Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 4

«ОРГАНИЗАЦИЯ АНАЛОГОВОГО ВВОДА-ВЫВОДА: АЦП И ШИМ»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Макаров А.С

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3
Ход работы 3-10
1. Теоретическая часть 3-6
2. Практическая часть 7-10
Ответы на контрольные вопросы 11-12
Вывод 13
Листинг «lab4.c-файл» 14-20

ЦЕЛЬ РАБОТЫ

Получение практических навыков по работе с аналого-цифровым преобразователем и таймерами-счётчиками. Изучение принципов применения широтно-импульсной модуляции в современных цифровых устройствах.

ХОД РАБОТЫ

1. Теоретическая часть

Аналого-цифровой преобразователь представляет собой аппаратный модуль, предназначенный для дискретизации непрерывного аналогового напряжения и преобразования его в цифровой код, пригодный для обработки микроконтроллером. В ATmega32 реализован 10-битный АЦП последовательного приближения, способный работать с восемью однополярными входными каналами, выведенными на порт PORTA (ADC0…ADC7).

Bit	7	6	5	4	3	2	1	0	
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

REFS1 и REFS0 – выбирают опорное напряжение

REFS1	REFS0	Voltage Reference Selection
0	0	AREF, Internal Vref turned off
0	1	AVCC with external capacitor at AREF pin
1	0	Reserved
1	1	Internal 2.56V Voltage Reference with external capacitor at AREF pin

00 – АЦП берёт напряжение с ножки AREF (мы сами подаём туда нужное напряжение)

01 – АЦП берёт опору от AVCC (питание микроконтроллера, например 5 В)

10 – Зарезервированный режим (не используется)

11 – АЦП использует внутреннее опорное напряжение 2.56V

ADLAR – отвечает за то, как результат АЦП будет лежать в регистрах ADCL и ADCH

MUXi – выбор канала АЦП (PA1,…,PA5)

Bit	7	6	5	4	3	2	1	0	
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ADCSRA управляет как запускать и контролировать АЦП.

ADEN – включатель АЦП

ADSC – кнопка начать измерение

ADATE – автозапуск АЦП по условию

ADIF – флаг «АЦП закончил измерение»

ADIE – разрешение прерывание АЦП

ADPS2, ADPS1, ADPS0 – биты делителя частоты АЦП

АЦП может работать в разных режимах. В простейшем случае программа запускает преобразование и ожидает его окончания. Такой способ называется программным опросом. Однако при этом микроконтроллер тратит время на ожидание результата. Более удобный способ – использование прерывания АЦП. В этом случае после завершения преобразования микроконтроллер автоматически переходит к обработчику прерывания, где можно считать результат измерения. Такой подход позволяет основной программе выполнять другие действия, пока АЦП производит преобразование.

Результат АЦП в ATmega32 является 10-битным, поэтому он не помещается в один 8-битный регистр. Для хранения результата используются два регистра: ADCL и ADCH. При правом выравнивании необходимо считывать оба регистра, сначала ADCL, затем ADCH. При левом выравнивании старшие биты результата оказываются в регистре ADCH, поэтому при невысоких требованиях к точности можно считывать только ADCH и получать 8-битное значение.

Для нормальной работы АЦП необходимо правильно выбрать частоту его тактирования. Если частота слишком высокая, преобразование выполняется быстрее, но точность результата может ухудшаться. Для получения стабильного результата используется делитель частоты. Он задаётся битами ADPS2, ADPS1 и ADPS0 в регистре ADCSRA. Делитель уменьшает частоту микроконтроллера до значения, подходящего для работы АЦП.

Широтно-импульсная модуляция используется для получения среднего значения напряжения с помощью цифрового сигнала. Микроконтроллер может выдавать на вывод только логический ноль или логическую единицу. Однако если быстро переключать вывод между этими состояниями, можно управлять средней мощностью, подаваемой на нагрузку. Для светодиода это воспринимается как изменение яркости.

Основным параметром ШИМ является скважность, то есть отношение времени активного уровня сигнала ко всему периоду. Если сигнал находится в активном состоянии малую часть периода, светодиод светит тускло. Если сигнал находится в активном состоянии большую часть периода, светодиод светит ярко. При скважности около 50% светодиод имеет примерно среднюю яркость.

Для формирования ШИМ-сигнала удобно использовать таймеры-счётчики микроконтроллера. Таймер автоматически считает от начального значения до верхней границы, после чего цикл повторяется. В режиме Fast PWM таймер формирует периодический сигнал, а момент переключения выхода определяется значением регистра сравнения OCR. Чем больше значение OCR, тем большую часть периода выход находится в активном состоянии.

Таймеры позволяют формировать ШИМ аппаратно, без постоянного участия основной программы. Это важно, потому что микроконтроллер может одновременно выполнять основную логику программы, обрабатывать

прерывания и обновлять индикацию, а таймер в это время самостоятельно поддерживает нужную форму сигнала на выходе.

Внешние прерывания используются для реакции на события, происходящие вне микроконтроллера, например нажатие кнопки. Вместо постоянного опроса состояния кнопки программа может разрешить внешнее прерывание. Тогда при изменении сигнала на соответствующем выводе микроконтроллер автоматически вызывает обработчик прерывания. Это позволяет быстро реагировать на нажатия и не нагружать основной цикл постоянными проверками.

Семисегментный индикатор состоит из семи светящихся сегментов и, как правило, дополнительной точки. Комбинируя включение отдельных сегментов, можно отображать цифры и некоторые буквы. Для вывода символа на индикатор используется код сегментов, в котором каждый бит соответствует отдельному сегменту.

Многоразрядные семисегментные индикаторы часто работают в режиме динамической индикации. При таком способе все разряды используют общие линии сегментов, а выбор конкретного разряда выполняется отдельными управляющими линиями. В каждый момент времени включается только один разряд, но переключение происходит достаточно быстро. Благодаря инерции зрения человек воспринимает изображение как одновременное отображение всех разрядов.

Циклический сдвиг используется для реализации эффекта «бегущего огня». При обычном сдвиге бит, вышедший за границу байта, теряется. При циклическом сдвиге этот бит возвращается с другой стороны. Благодаря этому набор горящих светодиодов может перемещаться по порту по кругу, не исчезая после достижения края.

2. Практическая часть

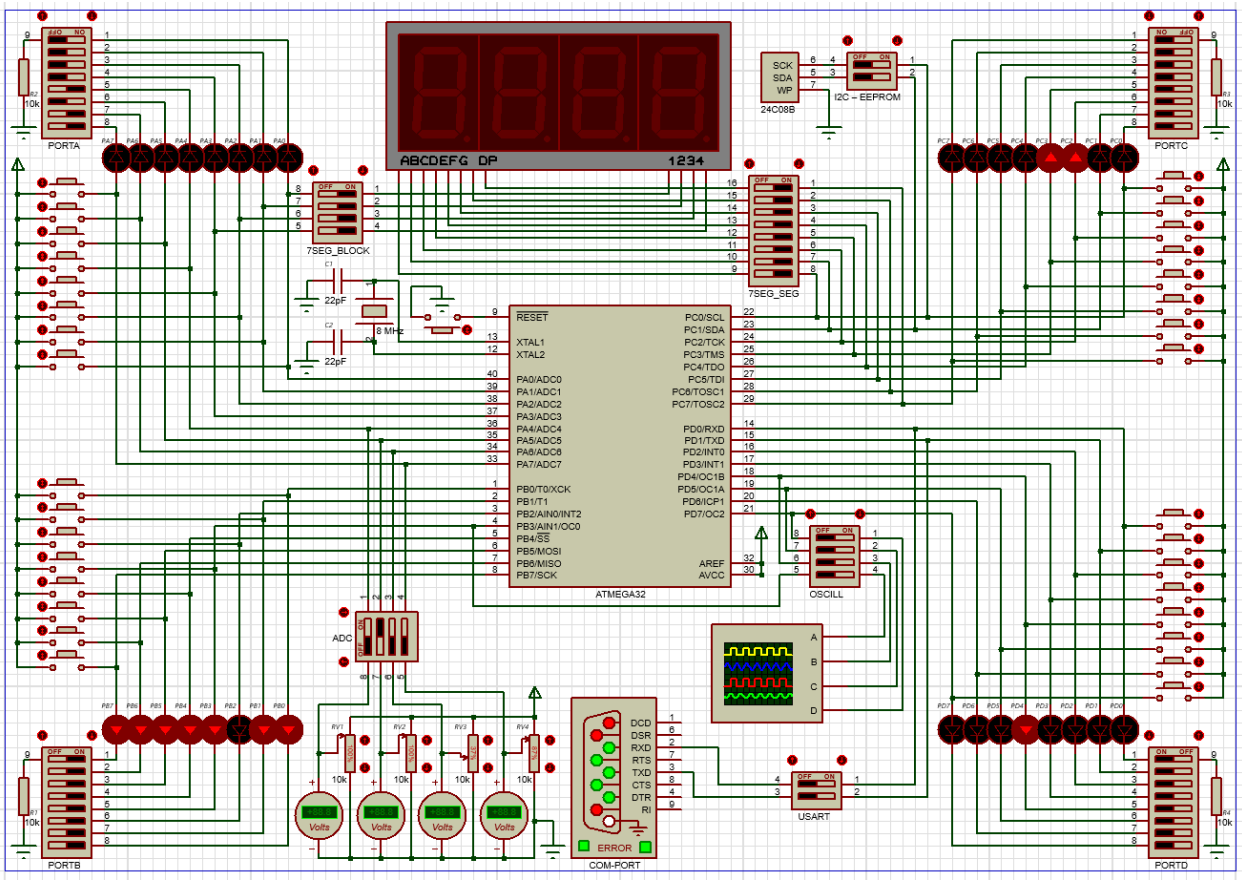


Рисунок 1 – Схема установки

После запуска программы выполняется начальная настройка микроконтроллера. Сначала настраиваются порты ввода-вывода: PORTA, PORTB и PORTC используются для отображения гирлянды, при этом вывод PA5 оставляется входом для АЦП. Вывод PD2 настраивается как вход для кнопки переключения режима работы, а выводы PD7 и PD4 используются как выходы ШИМ. Затем настраивается АЦП: выбирается канал ADC5, включается левое выравнивание результата, разрешается прерывание АЦП и задаётся делитель частоты. После этого настраивается внешнее прерывание INT0 для кнопки PD2, а также таймеры T1 и T2 для формирования ШИМ-сигналов.

После инициализации глобально разрешаются прерывания и запускается первое преобразование АЦП. Далее программа задаёт начальные состояния гирлянды на основе базовых значений, начальных смещений и

параметров сдвига. После этого программа переходит в бесконечный цикл, в котором работа зависит от значения переменной mode.

Если mode = 0, программа работает в режиме гирлянды. В этом режиме для каждого порта отдельно выполняется циклический сдвиг текущего состояния. На PORTA, PORTB и PORTC выводятся соответствующие битовые маски, определяющие, какие светодиоды должны гореть. Для каждого порта используется свой счётчик, поэтому состояния портов меняются с разной частотой. После каждого шага состояние порта циклически сдвигается на заданную величину.

Если mode = 1, программа переходит в режим настройки параметра. В данном варианте настраивается параметр bb, который задаёт базовое значение гирлянды для PORTB. Значение с АЦП пересчитывается в диапазон от 0 до 127 и записывается в переменную bb. Для наглядности текущее значение bb также выводится на светодиоды PORTB.

В режиме настройки дополнительно обновляется ШИМ. По текущему значению bb вычисляются значения регистров OCR2 и OCR1B, которые задают яркость светодиодов на выводах PD7 и PD4. Благодаря этому яркость светодиодов показывает, насколько параметр близок к нижней или верхней границе допустимого диапазона.

Также в режиме настройки работает семисегментный индикатор. На нём отображается имя настраиваемого параметра и его значение в шестнадцатеричной системе в формате bb.XX. Первые два разряда показывают имя параметра, а два последних – текущее значение. Значение периодически загорается и гаснет с интервалом около 0,5 с, что реализуется с помощью счётчика мигания.

Переключение между режимами выполняется по нажатию кнопки PD2. При срабатывании внешнего прерывания INT0 значение переменной mode изменяется на противоположное. Таким образом, программа циклически переходит из режима гирлянды в режим настройки и обратно. Значение

потенциометра считывается через прерывание АЦП, поэтому оно постоянно обновляется и используется при настройке параметра bb.

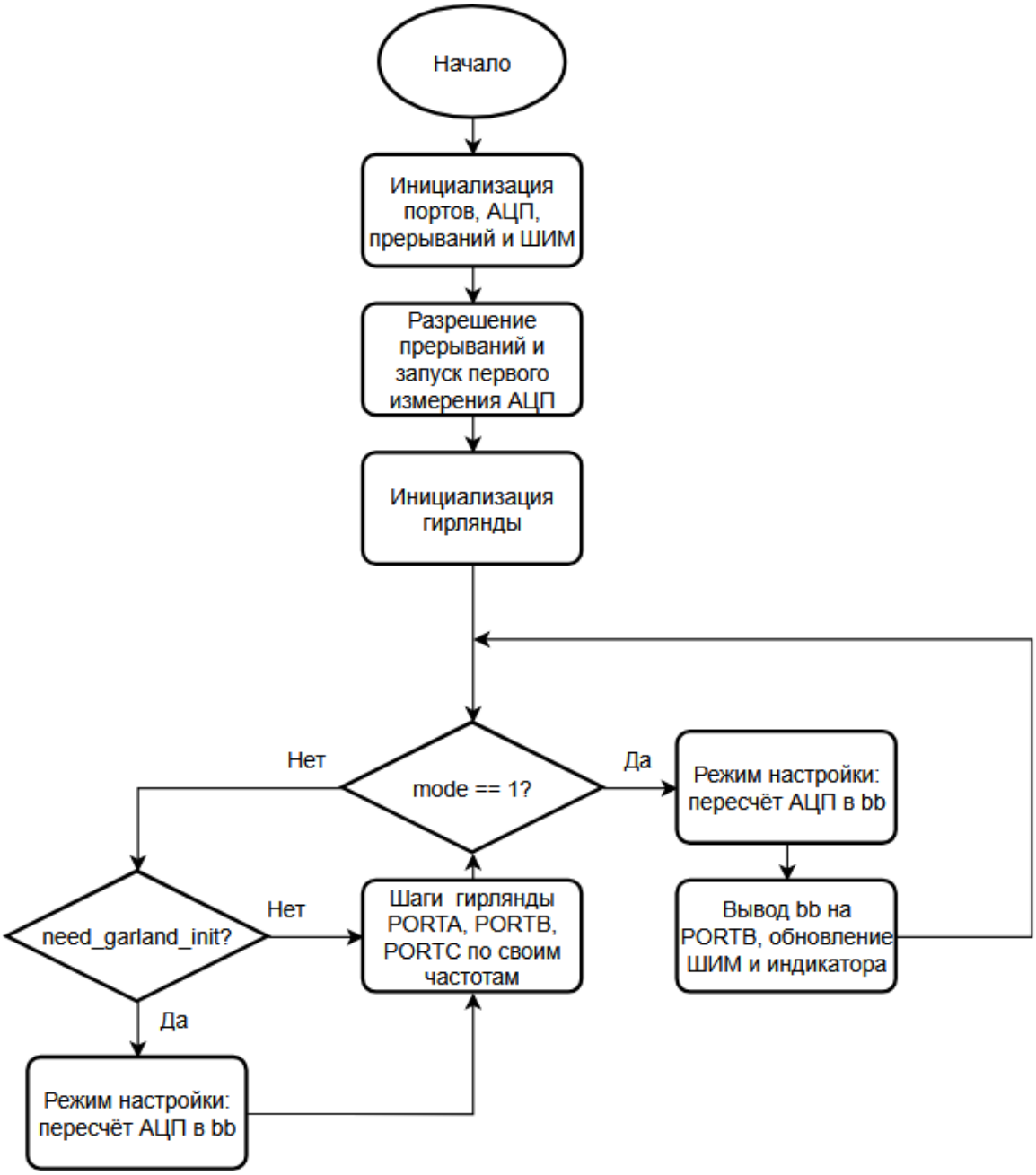


Рисунок 2 – Блок-схема алгоритма работы программы

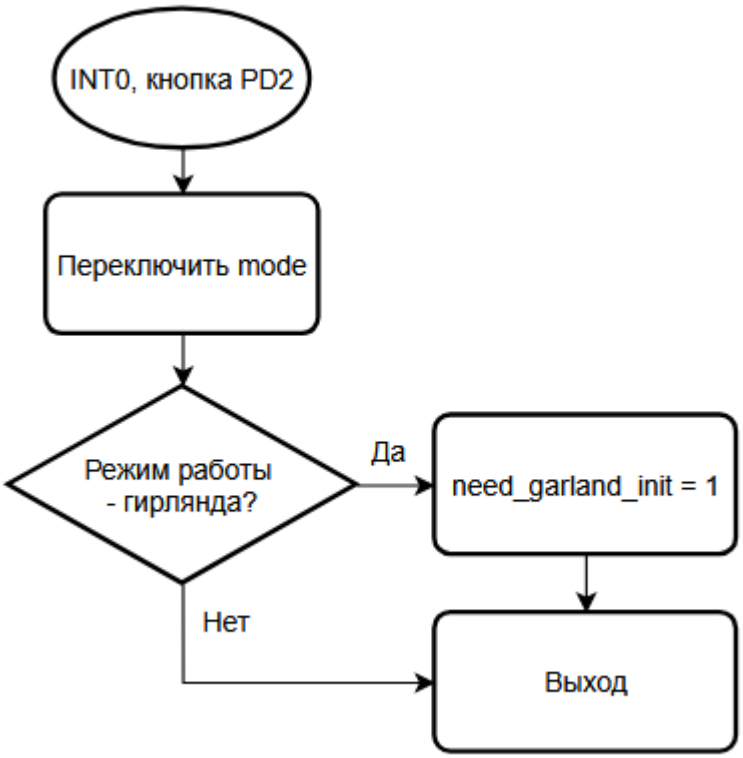


Рисунок 3 – Блок-схема обработки прерывания INT0

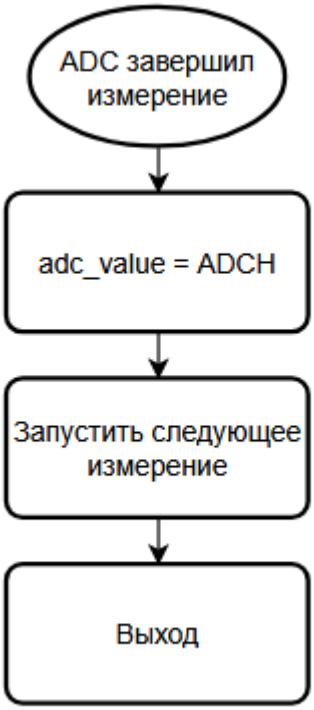


Рисунок 4 – Блок-схема обработчика прерывания АЦП

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1) Посредством каких регистров производится настройка АЦП?

Настройка АЦП производится с помощью регистров ADMUX и ADCSRA. Регистр ADMUX отвечает за выбор опорного напряжения, канала АЦП и способа выравнивания результата. Регистр ADCSRA используется для включения АЦП, запуска преобразования, разрешения прерываний и выбора делителя частоты АЦП. Результат преобразования хранится в регистрах ADCL и ADCH.

2) В каких режимах может работать АЦП?

АЦП может работать в режиме одиночного преобразования, когда каждое измерение запускается программно установкой бита ADSC. Также АЦП может работать в режиме автоматического запуска, когда преобразование начинается по определённому событию. Кроме того, возможен режим свободного запуска, при котором АЦП постоянно выполняет преобразования и обновляет результат. Считывание результата может выполняться опросом флага завершения или через прерывание АЦП.

3) Какие порты и разряды портов микроконтроллера ATmega32 могут обрабатывать входящие аналоговые сигналы?

В ATmega32 входящие аналоговые сигналы могут обрабатываться выводами порта PORTA, которые имеют функции каналов АЦП. Это выводы PA0...PA7, соответствующие каналам ADC0...ADC7. В данной работе используется канал ADC5, то есть вывод PA5, к которому подключается потенциометр.

4) Какими способами реализуется ШИМ?

ШИМ может быть реализована программным способом или аппаратным способом. При программной реализации микроконтроллер сам в программе переключает вывод между нулём и единицей с нужными задержками. При аппаратной реализации используется таймер-счётчик, который автоматически формирует ШИМ-сигнал на специальном выводе

микроконтроллера. Аппаратный способ удобнее, так как не нагружает основной цикл программы.

5) Как настроить ШИМ с помощью таймера-счётчика?

Для настройки ШИМ с помощью таймера-счётчика необходимо выбрать режим работы таймера, например Fast PWM, настроить вывод сравнения, выбрать делитель частоты и задать значение регистра сравнения OCR. Таймер считает от нуля до верхней границы, а значение OCR определяет скважность сигнала. Чем больше значение OCR, тем большую часть периода сигнал находится в активном состоянии, а значит тем выше яркость светодиода.

ВЫВОД

В ходе лабораторной работы были изучены принципы работы АЦП и ШИМ в микроконтроллере ATmega32. Была реализована программа с двумя режимами работы: режимом гирлянды и режимом настройки параметра. Для считывания положения потенциометра использовался АЦП, работающий через прерывание.

Также была реализована настройка параметра bb для гирлянды по варианту 5в. Значение параметра отображалось на семисегментном индикаторе в шестнадцатеричной системе. Для индикации положения параметра относительно границ диапазона использовалась ШИМ на выводах PD7 и PD4.

В результате были получены практические навыки настройки портов ввода-вывода, АЦП, внешних прерываний, таймеров и ШИМ в микроконтроллере ATmega32.

Листинг «lab4.c файл»

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h> // подключаем задержки

#define SEG_B 0x7C
#define SEG_DOT 0x80

uint16_t blink_counter = 0;
uint8_t value_visible = 1;

volatile uint8_t adc_value = 0;
volatile uint8_t need_garland_init = 0;

// Параметры
volatile uint8_t ba = 3;
volatile int8_t  ha = 2;
volatile uint8_t pa = 4;
volatile uint8_t da = 0;

volatile uint8_t bb = 0x0F; // Это настраиваем
volatile int8_t  hb = 1;
volatile uint8_t pb = 1;
volatile uint8_t db = 0;

volatile uint8_t bc = 6;
volatile int8_t  hc = 1;
volatile uint8_t pc = 6;
volatile uint8_t dc = 1;

volatile uint8_t mode = 0;           // 0 - гирлянда, 1 -
настройка

uint8_t state_a;
uint8_t state_b;
uint8_t state_c;

const uint8_t seg_hex[16] =
{
     0x3F, // 0
     0x06, // 1
     0x5B, // 2
     0x4F, // 3
     0x66, // 4
     0x6D, // 5
     0x7D, // 6
     0x07, // 7
     0x7F, // 8
     0x6F, // 9
     0x77, // A
     0x7C, // b
     0x39, // C
```

```
    0x5E, // d
    0x79, // E
    0x71  // F
};

ISR(INT0_vect) // Кнопка PD2: переключает режим работы программы
{
    mode ^= 1;

    if (mode == 0)
    {
        need_garland_init = 1; // вернулись в гирлянду
    }

}

void UpdateBlink(void) // Мигание значения на семисегментном
индикаторе
{
    blink_counter++;

    if (blink_counter >= 250) // 250 * 2 мс = 500 мс
    {
        blink_counter = 0;
        value_visible ^= 1;
    }
}

void DisplayDigit(uint8_t digit, uint8_t segments)
{
    PORTA &= 0xF0;       // выключаем PA0..PA3
    PORTC = ~segments;   // сегменты активны нулём

    if (digit == 0)
    {
        PORTA |= (1 << PA0);
    }
    else if (digit == 1)
    {
        PORTA |= (1 << PA1);
    }
    else if (digit == 2)
    {
        PORTA |= (1 << PA2);
    }
    else if (digit == 3)
    {
        PORTA |= (1 << PA3);
    }
}

void DisplaySetting(void)
{
```

```
     static uint8_t digit = 0; // номер текущего разряда
индикатора
     uint8_t high_nibble = (bb >> 4) & 0x0F; // Это старшая hex-
цифра числа bb
     uint8_t low_nibble = bb & 0x0F; // Это младшая hex-цифра
числа bb

     if (digit == 0)
     {
          DisplayDigit(0, SEG_B);
     }
     else if (digit == 1)
     {
          DisplayDigit(1, SEG_B | SEG_DOT);
     }
     else if (digit == 2)
     {
          DisplayDigit(2, value_visible ? seg_hex[high_nibble] :
0x00);
     }
     else if (digit == 3)
     {
          DisplayDigit(3, value_visible ? seg_hex[low_nibble] :
0x00);
     }

     digit++;

     if (digit >= 4)
     {
          digit = 0;
     }
}

void UpdateParameter(void)
{
     bb = adc_value / 2;
     //bb = ((uint16_t)adc_value * 127 + 127) / 255;
}

uint8_t RotateLeft(uint8_t value, uint8_t shift) // Функция
циклического сдвига влево
{
     shift %= 8;
     return (value << shift) | (value >> (8 - shift));
}

void GarlandInit(void)
{
     state_a = RotateLeft(ba, da);
     state_b = RotateLeft(bb, db);
     state_c = RotateLeft(bc, dc);
}
```

```
void GarlandStepA(void)
{
     PORTA = state_a & ~(1 << PA5);
     state_a = RotateLeft(state_a, ha);
}

void GarlandStepB(void)
{
     PORTB = state_b;
     state_b = RotateLeft(state_b, hb);
}

void GarlandStepC(void)
{
     PORTC = state_c;
     state_c = RotateLeft(state_c, hc);
}

void PortsInit(void) // Настройка портов
{
     DDRA = 0xDF; // PA5 – на вход для АЦП 11011111, т.е к нему
подключаем потенциометр для АЦП
     DDRB = 0xFF; // на выход для гирлянды
     DDRC = 0xFF; // на выход для гирлянды

     DDRD &= ~(1 << PD2); /* делаем PD2, PD3 – входами. PD2 -
переключает режим работы программы
                                                  режим гирлянды и
режим настройки.
                                                       PD3 – кнопка
которая работает в режиме настройки для переключения параметров
*/

     DDRD |= (1 << PD7) | (1 << PD4);    // делаем PD7, PD4 –
выходами (на них выводится ШИМ)
}

void UpdatePwm(void)
{
     OCR1B = (uint16_t)bb * 255 / 127;
     OCR2 = 255 - OCR1B;
}

void AdcInit(void) // Настройка АЦП - модуля, к-ый читает
напряжение с потенциометра и преобразует его в число
{
     ADMUX = (1 << REFS0) // Выбираем опорное напряжение 5V,
примерно питание платы, оно говорит какое напряжение считать
максимумом
     | (1 << ADLAR)       // Читаем только ADCH, для параметров
достаточно 0...255
     | 5;
```

```
     ADCSRA = (1 << ADEN) // Включаем АЦП
     | (1 << ADIE)        // Разрешение прерывание АЦП
     | (1 << ADPS2)       // Делитель частоты – 128 . Скорость
работы АЦП
     | (1 << ADPS1)
     | (1 << ADPS0);
}

void PwmInit(void)
{
     // Timer2: Fast PWM, выход OC2 на PD7.
     TCCR2 = (1 << WGM20) // TCCR2 - регистр настройки таймера 2.
     | (1 << WGM21) // WGM20=1, WGM21=1. То есть он быстро
считает от 0 до 255 и формирует ШИМ-сигнал. ШИМ-сигнал - это
быстрое включение/выключение.
     | (1 << COM21) // включить PWM-выход OC2 (PD7)
     | (1 << CS21); // делитель 8. Задаём скорость счёта таймера

     // Timer1: Fast PWM 8-bit, выход OC1B на PD4
     TCCR1A = (1 << WGM10)
     | (1 << COM1B1);

     TCCR1B = (1 << WGM12)
     | (1 << CS11); // делитель 8

     // Начальная яркость
     OCR2 = 0;  // PD7
     OCR1B = 0; // PD4
}

void InterruptsInit(void)
{
     // INT0 по нарастающему фронту: 0 -> 1
     MCUCR &= ~((1 << ISC01) | (1 << ISC00));
     MCUCR |= (1 << ISC01) | (1 << ISC00);

     GICR |= (1 << INT0);
}

int main(void)
{
     PortsInit(); // Настройка портов
     AdcInit();   // Настройка АЦП
     InterruptsInit(); // Настройка прерываний
     PwmInit();        // Настройка ШИМа

     sei();                   // глобально разрешить прерывания
     ADCSRA |= (1 << ADSC);  // первое измерение

     GarlandInit();
```

```
     uint8_t tick = 0; //Счётчик шагов времени
     while (1)
     {
          if (mode == 1) // настройка bb
          {
               UpdateParameter(); // Берём значение с АЦП и
пересчитываем его в параметр
               PORTB = bb;       // показываем текущее bb на
светодиодах PORTB
               UpdatePwm();
               UpdateBlink();
               DisplaySetting();

               _delay_ms(2);

          }
          else
          {

               if (need_garland_init)
               {
                    need_garland_init = 0;
                    GarlandInit();
               }

               if (tick % pa == 0) // pa=4 => раз в секунду
               {
                    GarlandStepA();
               }

               if (tick % pb == 0) // pb=1 => раз в 0.25 с.
               {
                    GarlandStepB();
               }

               if (tick % pc == 0) // pc=6 => раз в 1.5 с.
               {
                    GarlandStepC();
               }

               tick++;

               if (tick >= 60)
               {
                    tick = 0;
               }

               _delay_ms(250);
          }
     }
}
```

```
ISR(ADC_vect)                // После завершения измерения АЦП
программа перейдёт сюда. Обработчик прерывания АЦП
{
     adc_value = ADCH;       // забираем результат измерения
     ADCSRA |= (1 << ADSC);
}
```